

Guía de Uso de la Plataforma Aurora AI Studio

Manual de onboarding: creación de cuenta, tour del dashboard, primer proyecto RAG, invocación de la Chat API y mejores prácticas.

Aurora AI Studio S.A.S.

OCTUBRE 2025

1. Creación de Cuenta y Verificación

Para comenzar a utilizar Aurora AI Studio, diríjase a <https://aurora-ai.studio/signup> y complete el formulario de registro con su correo electrónico corporativo, una contraseña que cumpla las políticas mínimas (12 caracteres, mayúsculas, minúsculas, números y un símbolo) y el nombre de su organización. Recibirá un correo de verificación con un enlace de un solo uso válido por 24 horas; al hacer clic, su cuenta quedará activa con el plan Free, que incluye 10.000 tokens de regalo para probar la plataforma. Si no recibe el correo en cinco minutos, verifique la carpeta de spam o solicite reenvío desde la pantalla de login.

Para activar el acceso a las APIs en producción deberá completar la verificación de identidad: cargue un documento de identidad válido y un comprobante de representación legal si actúa en nombre de una empresa. La verificación se completa en un plazo máximo de dos (2) días hábiles. Mientras tanto, puede explorar el panel, crear proyectos y probar la plataforma desde el playground. La verificación es necesaria para generar API keys con cuotas de producción; las API keys de prueba tienen cuotas reducidas y marcas de agua en las respuestas.

2. Tour por el Dashboard

El dashboard es el centro de operaciones de la plataforma. Desde aquí se administran proyectos, API keys, consumos, facturación y soporte. La barra lateral izquierda organiza las secciones en cinco grupos: **Proyectos** (gestión de workspaces aislados), **Desarrollo** (API keys, playground, SDKs, logs), **Datos** (knowledge bases, datasets, modelos fine-tuned), **Facturación** (planes, consumo, facturas, métodos de pago) y **Configuración** (cuenta, organización, seguridad, integraciones). La barra superior muestra el proyecto activo, el plan actual, el consumo del mes y el acceso al centro de ayuda.

Cada proyecto es un workspace aislado con su propia base de conocimiento, API keys, miembros y configuración. Esta separación permite gestionar distintos entornos (desarrollo, staging, producción) o distintos clientes (si usted es consultor o agencia) sin mezclar datos ni costos. Para crear un proyecto, haga clic en el selector de proyecto (barra superior) → **"Nuevo proyecto"** → asigne un nombre y descripción. El proyecto se crea con una API key de prueba por defecto; puede generar más desde **Desarrollo** → **API Keys**.

3. Su Primer Proyecto RAG

Un proyecto RAG (Retrieval-Augmented Generation) permite construir un asistente que responde preguntas basadas en su propia documentación. El flujo básico consiste en: (1) cargar documentos a una knowledge base, (2) indexar el contenido en fragmentos vectoriales, (3) probar consultas en el playground, y (4) integrar el endpoint en su aplicación. Para crear su primer proyecto RAG, navegue a **Datos** → **Knowledge Bases** → **Nueva**, asigne un nombre, seleccione el modelo de embeddings (recomendado: *aurora-embed-3-small*) y la estrategia de chunking

(recomendado: 800 tokens con 100 tokens de overlap).

Una vez creada la knowledge base, cargue sus documentos. Aurora AI Studio soporta formatos PDF, DOCX, TXT, MD, CSV y HTML hasta 50 MB por archivo. Puede arrastrar y soltar archivos en la interfaz o usar el SDK de Python. Tras la carga, la plataforma procesa automáticamente los documentos: extrae texto, divide en fragmentos, genera embeddings y los almacena en la base vectorial. El tiempo de procesamiento depende del tamaño; documentos de hasta 100 páginas se procesan en menos de dos minutos.

Python

```
# Ejemplo: cargar documentos vía SDK Python
from aurora_sdk import AuroraClient
client = AuroraClient(api_key="sk-aurora-...")
kb = client.knowledge_bases.get("kb_intranet_v1")
# Subir múltiples documentos
kb.upload(["docs/manual-empleado.pdf", "docs/politica-vacaciones.docx", "docs/faq-clientes.md"])
# Verificar estado de indexación
status = kb.status()
print(status) # {"ready": True, "chunks": 1247, "tokens": 389421}
```

4. Invocar la Chat API

La Chat API permite enviar consultas en lenguaje natural y recibir respuestas generadas a partir del contexto recuperado de la knowledge base. El endpoint principal es **POST** <https://api.aurora-ai.studio/v1/chat/completions> y sigue una sintaxis compatible con OpenAI. Los parámetros clave son: **model** (recomendado *aurora-chat-pro*), **messages** (lista de mensajes con rol y contenido), **knowledge_base** (ID de la KB a consultar) y **citations** (booleano para incluir fuentes en la respuesta).

bash

```
# Ejemplo: invocar la Chat API con RAG curl
https://api.aurora-ai.studio/v1/chat/completions \
-H "Authorization: Bearer $AURORA_API_KEY" \
-H "Content-Type: application/json" \
-d '{ "model": "aurora-chat-pro", "knowledge_base": "kb_intranet_v1", "citations": true, "messages": [ { "role": "user", "content": "¿Cuántos días de vacaciones tengo por ley?" } ] }'
```

La respuesta incluye el contenido generado, las citas (archivo, sección y fragmento) y métricas de uso (tokens consumidos, latencia, modelo utilizado). Para aplicaciones de chat conversacional, incluya el historial completo de mensajes en cada request; Aurora AI Studio no mantiene estado entre llamadas por diseño. El límite de contexto por request es de 128.000 tokens para *aurora-chat-pro* y 32.000 para *aurora-chat-lite*.

5. Gestión de API Keys y Seguridad

Las API keys son credenciales sensibles: cualquier persona con acceso a la clave puede consumir el servicio a su cargo. Por eso, recomendamos seguir estrictamente estas prácticas: (i) nunca embeber API keys en código cliente (JavaScript de navegador, apps móviles descompilables); (ii) almacenar las claves en variables de entorno o gestores de secretos (Vault, AWS Secrets Manager, doppler); (iii) rotar las claves cada 90 días desde **Desarrollo** → **API Keys** → **Rotar**; (iv) crear claves distintas por entorno y por servicio para facilitar auditoría y revocación granular; (v) configurar límites de gasto por clave para prevenir sobrecostos por compromiso.

Aurora AI Studio permite restringir cada API key por dirección IP, por dominio referrer y por conjunto de endpoints autorizados. Estas restricciones se configuran al crear o editar la clave. Adicionalmente, la plataforma detecta patrones de uso anómalos (picos de consumo, geo-localización inusual) y puede pausar automáticamente una clave comprometida, notificando al correo del administrador. Ante cualquier sospecha de compromiso, revoque la clave inmediatamente desde el panel y genere una nueva.

6. Monitoreo de Uso y Costos

El panel **Facturación** → **Consumo** muestra en tiempo real el uso de cada servicio: tokens consumidos por modelo, requests por endpoint, almacenamiento vectorial y cómputo de fine-tuning. Puede filtrar por proyecto, por rango de fechas y por servicio. Configure alertas de gasto desde **Facturación** → **Alertas** para recibir notificaciones por correo o Slack cuando el consumo supere umbrales definidos (por ejemplo, 80% del cupo mensual). Las alertas ayudan a evitar sorpresas en la factura y a detectar bugs de código que consumen tokens en bucle.

Recomendación: habilite el "Hard limit" en su cuenta para que la plataforma detenga automáticamente las llamadas a la API cuando se alcance un monto máximo configurado. Esto evita cobros por encima del presupuesto y es especialmente útil en etapas de desarrollo.

7. Mejores Prácticas

Para sacar el máximo provecho de la plataforma, siga estas mejores prácticas validadas por nuestro equipo de soluciones. Están orientadas a optimizar costos, mejorar la calidad de las respuestas y mantener la seguridad operativa. Implementarlas desde el inicio evita retrabajo y deudas técnicas.

- Use el modelo *aurora-chat-lite* para tareas simples (clasificación, extracción) y reserve *aurora-chat-pro* para razonamiento complejo. Puede ahorrar hasta 70% en costos.
- Diseñe prompts system claros y concisos, especificando rol, formato de salida deseado y restricciones. Un buen prompt system reduce iteraciones y mejora consistencia.
- Solicite siempre **citations: true** cuando use RAG; permite validar la respuesta contra el fragmento fuente y reduce el riesgo de alucinaciones.
- Implemente cache de respuestas para consultas idénticas frecuentes; la plataforma soporta cache del lado del cliente y del servidor (parámetro *cache_ttl*).
- Mantenga su knowledge base actualizada; reindexe los documentos cuando haya cambios sustanciales. La plataforma notifica cuando un documento cambia en el storage conectado.
- Habilite el logging de prompts en entornos de desarrollo para depurar respuestas; desactívelo en producción para minimizar retención de datos sensibles.